

API Traffic Report

Using any language and runtime you're comfortable with, design and build a solution for the following problem. Budget about **one hour**. Using AI tools is permitted and not penalized.

The problem

You're given a log file of API requests, one JSON object per line (`JSONL`). Write a command-line program that reads the log file and prints a single JSON API traffic report to stdout.

INPUT

Each line looks like this:

```
{"request_id":"a1_1","timestamp":"2024-01-15T10:00:00Z","client_id":"acct_1","endpoint":"/v1/widgets","status_code":200}
```

Fields:

FIELD	TYPE	MEANING
<code>request_id</code>	string	Unique ID for the request.
<code>timestamp</code>	string	ISO-8601 / RFC 3339 timestamp (may use <code>Z</code> or a <code>+HH:MM</code> offset).
<code>client_id</code>	string	Who made the request.
<code>endpoint</code>	string	Path that was called.
<code>status_code</code>	integer	HTTP status code.

Log files are produced by a variety of third party upstream services owned by other teams. This program will run in production as a platform-owned service for API request observability.

FEATURES

Your API Traffic Report program should:

- Identify clients who violate reasonable HTTP rate limits
- Provide visibility into request counts
- Discard and count the number of malformed inputs

Use your best judgement to fill in any product requirement gaps.

Example input

Use the provided input to sanity-check your solution.

We will also run your program against a set of input files with the same command-line contract.

What to submit

1. Your solution, with a `README.md` covering:
 - The exact command to run it (language/runtime version included).
 - Solution design
 - What the program does, end to end
 - The rate-limit rule(s) you chose and why
 - Specification of the output produced
 - Implementation decisions and assumptions made (and why)
 - What you'd do differently with more time.
 - Whether/how you used AI tools.
2. Push it to a GitHub repo and send us the link (ensure we have permission to see it).
 - If you can't use GitHub for this, email us and we'll work out another way.

APPENDIX

sample_input/requests.jsonl

The sample log referenced above. Save this as a file and run your program against it.

```
{"request_id":"a1_1","timestamp":"2024-01-15T10:00:00Z","client_id":"acct_1","endpoint":"/v1/widgets","status_code":200}
{"request_id":"a1_2","timestamp":"2024-01-15T10:00:02Z","client_id":"acct_1","endpoint":"/v1/widgets","status_code":200}
{"request_id":"a1_3","timestamp":"2024-01-15T10:00:04Z","client_id":"acct_1","endpoint":"/v1/widgets","status_code":200}
{"request_id":"a1_4","timestamp":"2024-01-15T10:00:05Z","client_id":"acct_1","endpoint":"/v1/widgets","status_code":200}
{"request_id":"a1_5","timestamp":"2024-01-15T10:00:06Z","client_id":"acct_1","endpoint":"/v1/widgets","status_code":200}
{"request_id":"a1_6","timestamp":"2024-01-15T10:00:08Z","client_id":"acct_1","endpoint":"/v1/widgets","status_code":200}
{"request_id":"a2_1","timestamp":"2024-01-15T10:05:00Z","client_id":"acct_2","endpoint":"/v1/reports","status_code":200}
{"request_id":"a2_2","timestamp":"2024-01-15T10:05:05Z","client_id":"acct_2","endpoint":"/v1/reports","status_code":200}
```